

Desarrollo de una plataforma de análisis de información biomédica

Ruta práctica desde los fundamentos hasta una arquitectura multimodal

2026-07-31

Tabla de contenidos

Propósito del tutorial	1
Por qué se necesita una plataforma biomédica	2
Cómo estudiar esta ruta	5
El proyecto integrador: BioForge Analytics	6
Antes del código: definir el problema	9
Nivel 0. Del problema a un entorno reproducible	9
Nivel 1. Construir el núcleo común	11
Nivel 2. Integrar y analizar datos tabulares	14
Nivel 3. Trabajar con series de tiempo	16
Nivel 4. Procesar señales biomédicas	18
Nivel 5. Procesar imágenes biomédicas	20
Nivel 6. Integrar la plataforma	22
Nivel 7. Preparar la plataforma para operar	26
Nivel 8. Analítica avanzada e inteligencia artificial	28
Tendencias actuales y criterios de adopción	31
Efectividad y eficiencia del desarrollador	32
Plan de estudio de 40 semanas	33
Portafolio que demuestra capacidad	33
Lista final de verificación	34
Fuentes y documentación recomendada	35
Cierre	36

En un proyecto biomédico es frecuente encontrar los datos repartidos entre hojas de cálculo, archivos de señales, carpetas de imágenes y cuadernos de análisis. Cada fuente puede funcionar por separado, pero la fragmentación dificulta responder preguntas básicas: ¿de dónde salió este resultado?, ¿qué transformación recibió el dato?, ¿se puede repetir el análisis?, ¿las unidades son correctas?, ¿se utilizó la misma versión del algoritmo?

Una plataforma de análisis es necesaria para convertir esa colección dispersa en un proceso organizado. Su valor no depende de tener una interfaz llamativa, sino de integrar datos heterogéneos, validar su calidad, conservar la trazabilidad y permitir que otras personas reproduzcan los resultados. En el contexto biomédico, estas capacidades son indispensables porque una interpretación puede cambiar cuando se altera la frecuencia de muestreo de una señal, el orden temporal de una serie, la escala física de una imagen o la definición de una variable clínica.

En este tutorial construiremos **BioForge Analytics**, una plataforma educativa para analizar tablas, series de tiempo, señales e imágenes biomédicas. El proyecto crecerá desde un programa local hasta una arquitectura modular, segura, observable e interoperable. Cada nivel agregará una capacidad verificable al mismo sistema.

Propósito del tutorial

Esta guía propone una ruta de desarrollo de software desde principiante hasta avanzado. La programación es una parte del proceso, pero no es el punto de partida. Primero se define la necesidad, luego se estructura la

información y finalmente se seleccionan las herramientas.

Al finalizar la ruta, el estudiante estará en capacidad de:

- traducir una necesidad de investigación o apoyo asistencial en requisitos verificables;
- distinguir datos tabulares, series de tiempo, señales e imágenes;
- diseñar contratos de datos y metadatos comunes para las cuatro modalidades;
- construir procesos reproducibles de ingestión, validación, transformación y análisis;
- desarrollar componentes desacoplados mediante Python, pruebas y control de versiones;
- exponer capacidades de la plataforma mediante una API documentada;
- gestionar archivos grandes sin almacenarlos de forma inadecuada en una base de datos relacional;
- incorporar privacidad, seguridad, auditoría y observabilidad desde el diseño;
- reconocer el papel de HL7 FHIR y DICOMweb en la interoperabilidad en salud;
- evaluar modelos de inteligencia artificial sin confundir rendimiento computacional con utilidad clínica; y
- decidir cuándo una tecnología mejora la plataforma y cuándo solo agrega complejidad.

! Alcance educativo y responsabilidad

La plataforma de este tutorial utiliza datos sintéticos, abiertos o debidamente anonimizados. Los ejemplos no diagnostican, no recomiendan tratamientos y no deben conectarse a historias clínicas reales. Cualquier uso clínico exige validación con profesionales de salud, gestión formal de riesgos, gobierno institucional de datos, evaluación de factores humanos y cumplimiento de la regulación aplicable.

Por qué se necesita una plataforma biomédica

El problema no es la falta de algoritmos

Existen bibliotecas para leer archivos, filtrar señales, procesar imágenes y entrenar modelos. Sin embargo, un algoritmo aislado no resuelve la gestión del análisis. En la práctica, los problemas más frecuentes aparecen antes y después del modelo:

- nombres de variables que cambian entre archivos;
- fechas sin zona horaria o con formatos incompatibles;
- unidades que no se registran o se mezclan;
- frecuencias de muestreo desconocidas;
- imágenes sin escala física o con metadatos incompletos;
- identificadores personales dentro de archivos de trabajo;
- transformaciones manuales que no quedan documentadas;
- resultados imposibles de reproducir;
- modelos evaluados con particiones que mezclan información de un mismo participante; y
- análisis que funcionan en un equipo, pero no pueden ejecutarse en otro.

La plataforma debe reducir estos riesgos mediante reglas explícitas, no mediante confianza en la memoria de quien realizó el análisis.

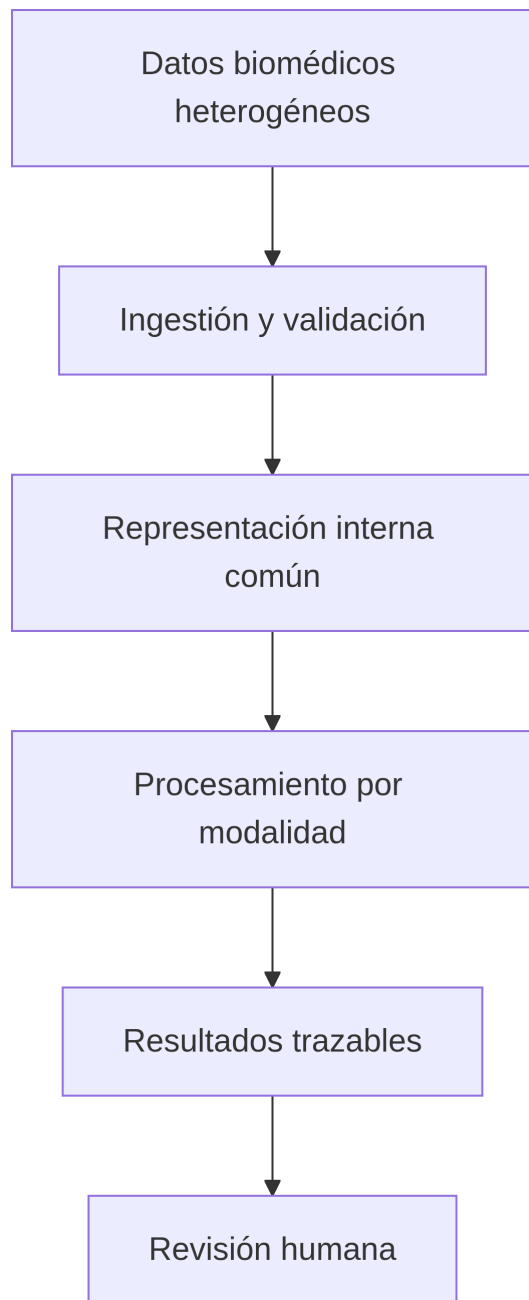
Cuatro modalidades, cuatro estructuras

Modalidad	Ejemplos	Estructura principal	Riesgo técnico frecuente
Tablas	datos demográficos, laboratorios, escalas, eventos	filas y columnas	tipos, categorías, unidades o faltantes inconsistentes
Series de tiempo	presión diaria, actividad por minuto, evolución de una variable	observaciones ordenadas por tiempo	marcas temporales duplicadas, irregulares o sin zona horaria

Modalidad	Ejemplos	Estructura principal	Riesgo técnico frecuente
Señales	ECG, EEG, EMG, PPG	amplitud muestreada a alta frecuencia	frecuencia de muestreo errónea, filtrado inadecuado o artefactos
Imágenes	radiografía, CT, MRI, ultrasonido	matriz 2D, 3D o 4D con metadatos espaciales	pérdida de escala física, orientación, profundidad de bits o procedencia

Una señal también es una serie temporal, pero conviene tratarlas como modalidades distintas. Una serie clínica puede tener una medición por día y admitir métodos para datos irregulares. Un ECG puede contener cientos de muestras por segundo y exige conocer la frecuencia de muestreo, el sistema de adquisición y el efecto de cada filtro.

Capacidades que la plataforma debe garantizar



Capacidad	Pregunta que debe responder
Calidad	¿El dato cumple el contrato definido para su modalidad?
Trazabilidad	¿Cuál fue el origen, la versión y la secuencia de transformaciones?
Reproducibilidad	¿Otra persona puede obtener el mismo resultado?
Extensibilidad	¿Se puede agregar una modalidad o algoritmo sin reescribir el sistema?
Privacidad	¿Se recopila y expone solo la información necesaria?

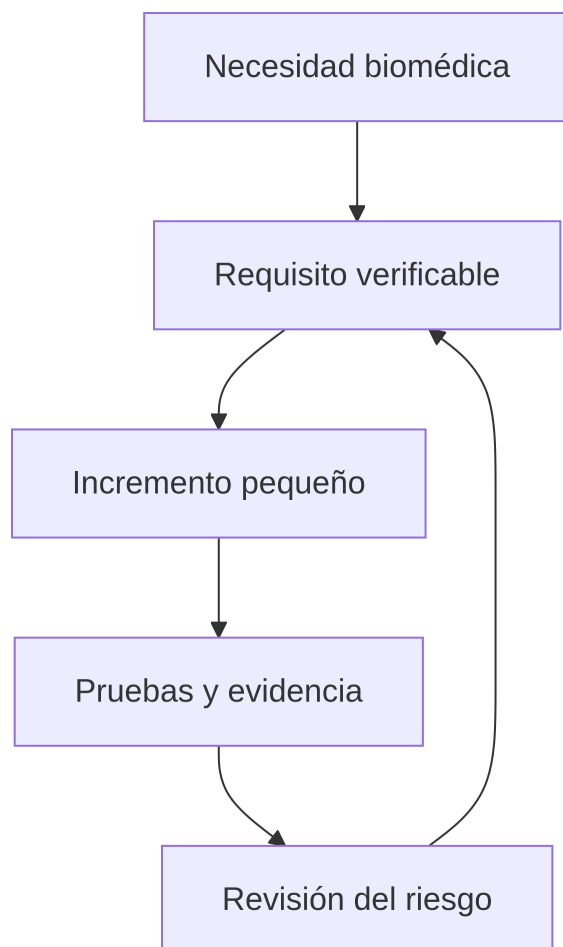
Capacidad	Pregunta que debe responder
Seguridad	¿Cada persona y servicio realiza únicamente acciones autorizadas?
Observabilidad	¿Se puede detectar dónde y por qué falló un proceso?
Interoperabilidad	¿La información puede intercambiarse con significado y contexto?
Validez	¿La evidencia respalda el uso específico que se propone?

Principio central: el resultado de un análisis biomédico no es solo una cifra, una gráfica o una máscara. También incluye los datos de entrada, sus metadatos, la versión del código, los parámetros, las advertencias de calidad y la evidencia de validación.

Cómo estudiar esta ruta

La ruta está organizada en **40 semanas**, con una dedicación aproximada de **8 horas por semana**. Puede ajustarse a un semestre extendido, un semillero o un proyecto de grado. Si se dispone de menos tiempo, conviene reducir el alcance funcional, no eliminar las pruebas o la documentación.

El ciclo de trabajo será siempre el mismo:



Una distribución semanal razonable es la siguiente:

Actividad	Proporción	Evidencia esperada
Estudiar el problema y el concepto	20 %	notas breves, supuestos y preguntas concretas
Implementar un incremento	45 %	código pequeño que funciona
Probar y revisar	25 %	resultados de pruebas y defectos corregidos
Documentar decisiones	10 %	cambios, límites y siguiente paso

La regla de avance es sencilla: **cada concepto debe producir una evidencia**. Puede ser una prueba automatizada, un conjunto de datos validado, una decisión de arquitectura, una medición de rendimiento o una demostración reproducible.

El proyecto integrador: BioForge Analytics

BioForge Analytics será una plataforma educativa de análisis biomédico multimodal. Su primera versión apoyará a un grupo de investigación que trabaja con datos tabulares de participantes, mediciones longitudinales, segmentos de ECG e imágenes médicas sintéticas.

La plataforma permitirá:

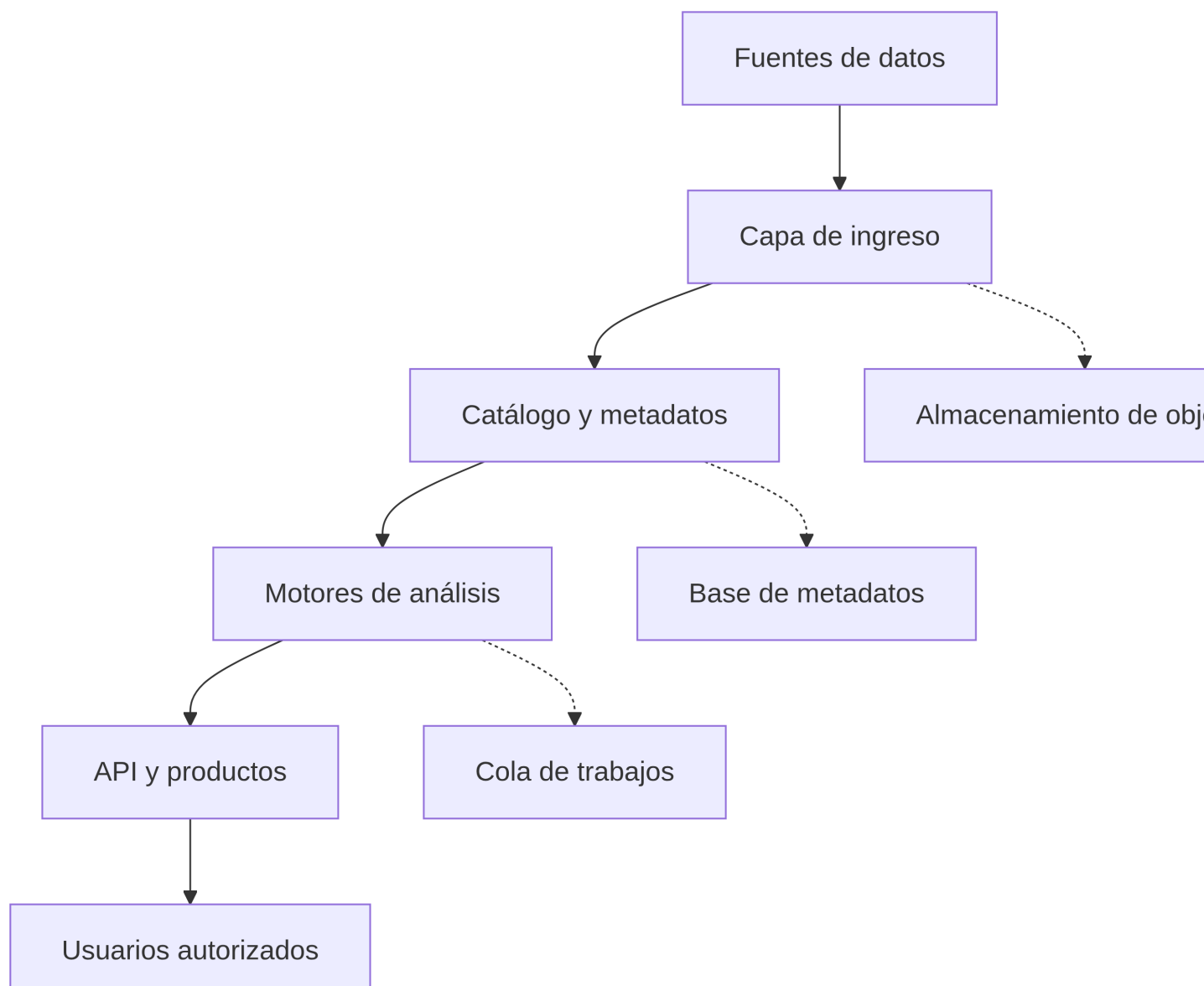
- registrar conjuntos de datos mediante identificadores anónimos;
- cargar y validar archivos sin modificar los originales;
- normalizar metadatos comunes;
- ejecutar procesos específicos para cada modalidad;
- generar estadísticas, características, visualizaciones y artefactos derivados;
- conservar el origen de cada resultado;
- consultar el estado de los procesos mediante una API;
- registrar eventos técnicos sin incluir información sensible; y
- preparar salidas interoperables cuando el caso de uso lo requiera.

Lo que no será la plataforma

BioForge Analytics no será:

- una historia clínica electrónica;
- un dispositivo médico;
- un sistema autónomo de diagnóstico;
- un repositorio de datos personales sin gobierno institucional;
- un único cuaderno de Jupyter con todas las funciones; ni
- una colección de microservicios creada antes de demostrar que son necesarios.

Arquitectura conceptual



La arquitectura separa tres elementos que suelen confundirse:

1. **Los archivos originales** se conservan sin modificación en almacenamiento de objetos o en un sistema de archivos controlado.
2. **Los metadatos** se guardan en una base estructurada para buscar, relacionar y auditar conjuntos de datos.
3. **Los artefactos derivados** contienen resultados reproducibles: tablas limpias, señales filtradas, imágenes transformadas, métricas, modelos o informes.

Contrato mínimo de información

Todas las modalidades compartirán un manifiesto. Este documento no reemplaza los metadatos específicos; permite reconocer los elementos comunes.

```
{  
  "dataset_id": "ds_ecg_001",
```



```

"subject_id": "sub_7f23a1",
"modality": "signal",
"format": "csv",
"source_uri": "raw/ecg/ds_ecg_001.csv",
"acquired_at": "2026-07-15T14:30:00-05:00",
"checksum": "sha256:...",
"schema_version": "1.0.0",
"attributes": {
  "sampling_rate_hz": 500,
  "channels": ["I", "II"]
}
}

```

Campo	Función
dataset_id	identifica el conjunto sin depender del nombre del archivo
subject_id	vincula datos de un participante seudonimizado
modality	selecciona el validador y el motor apropiados
format	indica cómo debe leerse el contenido
source_uri	localiza el original sin incrustarlo en la base de metadatos
acquired_at	conserva el momento de adquisición con zona horaria
checksum	permite comprobar que el archivo no cambió
schema_version	identifica las reglas usadas para interpretar el manifiesto
attributes	contiene metadatos particulares de la modalidad

Ruta acumulativa del proyecto

Nivel	Semanas	Incremento	Evidencia principal
0. Problema y entorno	1–2	alcance, repositorio y entorno reproducible	otra persona puede iniciar el proyecto
1. Núcleo común	3–6	modelos, contratos y registro de conjuntos	entradas válidas e inválidas se distinguen
2. Tablas	7–11	validación, limpieza y análisis tabular	informe de calidad reproducible
3. Series de tiempo	12–16	orden temporal, remuestreo y ventanas	no se pierde la semántica del tiempo
4. Señales	17–21	filtrado, segmentación y características	la transformación conserva parámetros
5. Imágenes	22–26	lectura, escala espacial y derivados	metadatos espaciales permanecen trazables
6. Integración	27–31	API, persistencia y trabajos asíncronos	flujo completo para las cuatro modalidades
7. Operación segura	32–36	pruebas, seguridad, despliegue y observabilidad	fallo reproducible y diagnosticable
8. Analítica avanzada	37–40	modelos, monitoreo e interoperabilidad	evaluación ligada a un uso definido

Antes del código: definir el problema

Una plataforma no se justifica porque pueda procesar muchos formatos. Se justifica cuando reduce un problema concreto de investigación, ingeniería o atención.

Para BioForge Analytics se define la siguiente necesidad:

Un grupo de investigación necesita analizar información biomédica multimodal sin perder la relación entre el archivo original, las transformaciones aplicadas y el resultado obtenido.

Preguntas de alcance

1. ¿Quién utilizará la plataforma?
2. ¿Qué decisiones o tareas apoyará?
3. ¿Qué modalidades son necesarias en la primera versión?
4. ¿Qué datos no deberían recopilarse?
5. ¿Qué volumen y velocidad de información se esperan?
6. ¿Qué puede ocurrir si un proceso falla o produce un resultado incorrecto?
7. ¿Qué evidencia demostraría que la plataforma funciona?
8. ¿Quién conserva la responsabilidad sobre la interpretación final?

Requisitos iniciales

Requisito funcional: la plataforma debe registrar un conjunto de datos mediante un manifiesto que incluya identificador, modalidad, formato, ubicación, fecha de adquisición, versión del esquema y suma de verificación.

Criterio de aceptación: si el manifiesto cumple el contrato y el archivo existe, el sistema debe registrarlo; si falta un campo obligatorio o la suma no coincide, debe rechazarlo y explicar el motivo.

El requisito expresa qué debe ocurrir. El criterio de aceptación permite comprobarlo. La implementación describe cómo se logró.

Requisitos no funcionales

Dimensión	Requisito educativo verificable
Privacidad	los ejemplos no almacenan nombres, documentos ni direcciones
Integridad	cada archivo tiene una suma SHA-256
Rendimiento	el registro de metadatos tarda menos de 500 ms en el entorno de prueba
Usabilidad	cada error identifica el campo o archivo que debe corregirse
Reproducibilidad	cada ejecución registra versión de código y parámetros
Mantenibilidad	los motores por modalidad pueden probarse de forma independiente
Trazabilidad	cada artefacto derivado referencia su entrada y su proceso
Disponibilidad	una copia de respaldo permite reconstruir el catálogo

Nivel 0. Del problema a un entorno reproducible

Meta: iniciar un proyecto que pueda instalarse y ejecutarse en otro equipo.

Duración sugerida: semanas 1 y 2.

Herramientas mínimas

Herramienta	Uso en la plataforma	Criterio de selección
Python	núcleo de procesamiento	ecosistema amplio para datos, señales, imágenes y web
uv	entornos y dependencias	instalación rápida y archivo de bloqueo reproducible
Git	historial y colaboración	permite revisar, comparar y recuperar cambios
Quarto	documentación	integra narrativa, código, tablas y diagramas
pytest	pruebas	convierte requisitos técnicos en controles repetibles
Ruff y Pyright	calidad estática	detectan problemas antes de ejecutar la plataforma

Se utilizará Python 3.13 como base estable. Una versión más reciente puede adoptarse cuando las bibliotecas científicas del proyecto hayan confirmado su compatibilidad.

Crear el proyecto

```
mkdir bioforge-analytics
cd bioforge-analytics

git init
uv init --python 3.13

uv add pydantic pydantic-settings typer rich
uv add pandas pyarrow pandera duckdb
uv add scipy xarray pywavelets
uv add pydicom simpleitk scikit-image
uv add fastapi "uvicorn[standard]" sqlalchemy alembic
uv add --dev pytest pytest-cov ruff pyright httpx

uv lock
uv run python --version
```

La documentación oficial de [uv](#) contiene métodos de instalación para Linux, macOS y Windows. Antes de ejecutar un instalador, se debe verificar su procedencia.

Estructura inicial

```
bioforge-analytics/
├── src/
│   └── bioforge/
│       ├── domain/           # reglas y modelos independientes
│       ├── application/      # casos de uso
│       ├── modalities/
│       │   ├── tabular/
│       │   └── timeseries/
```

```

    signals/
    images/
    infrastructure/      # base de datos, archivos y colas
    interfaces/          # API y línea de comandos
tests/
  unit/
  integration/
  fixtures/
data/
  sample/               # datos sintéticos pequeños
  README.md
docs/
  decisions/            # decisiones de arquitectura
  validation/
pyproject.toml
uv.lock
.gitignore
README.md

```

Los datos reales o voluminosos no deben guardarse en Git. El repositorio conserva código, esquemas, pruebas y muestras sintéticas pequeñas.

Primera decisión de arquitectura

Cree docs/decisions/0001-monolito-modular.md:

```

# ADR-0001: iniciar con un monolito modular

## Contexto
El equipo es pequeño y todavía está validando los flujos de análisis.

## Decisión
Usar una sola aplicación desplegable, separada internamente por dominio,
casos de uso, modalidades e infraestructura.

## Consecuencias
La operación inicial será sencilla. Los límites entre módulos deberán
mantenerse mediante interfaces y pruebas para permitir una separación futura.

```

Los microservicios no son una señal automática de madurez. Antes de dividir la plataforma se necesita evidencia de límites claros, equipos independientes, cargas diferentes o necesidades de despliegue separadas.

Criterio para avanzar

Otra persona debe poder clonar el repositorio, ejecutar `uv sync` y correr `uv run pytest` sin instalar dependencias manualmente.

Nivel 1. Construir el núcleo común

Meta: representar conjuntos de datos y procesos mediante modelos claros, sin depender todavía de una interfaz web.

Duración sugerida: semanas 3 a 6.

Conceptos de programación necesarios

- variables, tipos y operadores;
- funciones pequeñas con entradas y salidas explícitas;
- listas, diccionarios, conjuntos y tuplas;
- condiciones, ciclos y comprensión de colecciones;
- módulos, paquetes e importaciones;
- excepciones y mensajes de error;
- clases cuando representan conceptos estables del dominio;
- tipos opcionales, enumeraciones y protocolos; y
- lectura de archivos sin mezclarla con las reglas del dominio.

Modelo del manifiesto

```
from datetime import datetime
from enum import StrEnum
from typing import Any

from pydantic import BaseModel, Field, HttpUrl

class Modality(StrEnum):
    TABULAR = "tabular"
    TIMESERIES = "timeseries"
    SIGNAL = "signal"
    IMAGE = "image"

class DatasetManifest(BaseModel):
    dataset_id: str = Field(pattern=r"^ds_[a-z0-9_]+$")
    subject_id: str = Field(pattern=r"^sub_[a-z0-9]+$")
    modality: Modality
    format: str
    source_uri: str
    acquired_at: datetime
    checksum: str = Field(pattern=r"^sha256:[a-f0-9]{64}$")
    schema_version: str = "1.0.0"
    attributes: dict[str, Any] = Field(default_factory=dict)
```

Pydantic valida la forma del manifiesto. Las reglas que requieren consultar archivos, catálogos o permisos deben permanecer en servicios separados.

Verificar la integridad del archivo

```
from hashlib import sha256
from pathlib import Path

def calculate_sha256(path: Path, chunk_size: int = 1024 * 1024) -> str:
    digest = sha256()

    with path.open("rb") as source:
        while chunk := source.read(chunk_size):
            digest.update(chunk)
```

```

    return f"sha256:{digest.hexdigest()}"

def verify_file(path: Path, expected_checksum: str) -> None:
    if not path.is_file():
        raise FileNotFoundError(f"No existe el archivo: {path.name}")

    actual = calculate_sha256(path)
    if actual != expected_checksum:
        raise ValueError("El contenido no coincide con la suma registrada")

```

La lectura por bloques evita cargar un archivo completo en memoria. Esta decisión será importante cuando se trabajen estudios de imágenes o señales extensas.

Primera prueba automatizada

```

from hashlib import sha256

from bioforge.domain.integrity import calculate_sha256

def test_calculate_sha256_for_known_content(tmp_path):
    file_path = tmp_path / "sample.bin"
    content = b"bioforge"
    file_path.write_bytes(content)

    expected = f"sha256:{sha256(content).hexdigest()}"

    assert calculate_sha256(file_path) == expected

```

Una prueba útil no solo ejecuta código. Define un comportamiento esperado y falla de forma comprensible cuando el comportamiento cambia.

Registro de procedencia

Cada proceso debe producir un registro mínimo:

```

from datetime import UTC, datetime
from pydantic import BaseModel

class ProcessRecord(BaseModel):
    process_id: str
    input_dataset_ids: list[str]
    output_artifact_ids: list[str]
    algorithm: str
    algorithm_version: str
    parameters: dict[str, object]
    code_revision: str
    started_at: datetime
    finished_at: datetime | None = None
    status: str = "pending"

```

```
def utc_now() -> datetime:
    return datetime.now(UTC)
```

No guarde datos biomédicos sensibles dentro de los registros técnicos. Use identificadores y referencias controladas.

Criterio para avanzar

El núcleo debe aceptar un manifiesto válido, rechazar uno inválido, verificar la integridad de un archivo y registrar la procedencia de un proceso mediante pruebas automatizadas.

Nivel 2. Integrar y analizar datos tabulares

Meta: construir un proceso tabular que valide el esquema antes de calcular resultados.

Duración sugerida: semanas 7 a 11.

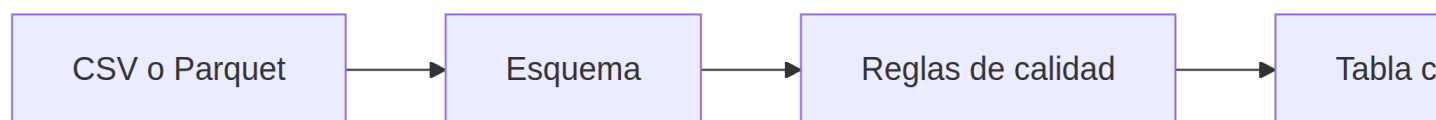
Caso de uso

El grupo recibe una tabla con mediciones sintéticas:

subject_id	visit	parameter	value	unit	measured_at
sub_a31f	1	heart_rate	72	bpm	2026-07-01T08:00:00-05:00
sub_a31f	2	heart_rate	75	bpm	2026-07-08T08:00:00-05:00

Antes de calcular promedios se debe comprobar que las columnas existen, los tipos son válidos, las unidades son coherentes y cada observación tiene significado.

Flujo tabular



Validar con Pandera

```
import pandas as pd
import pandera.pandas as pa
from pandera import Check, Column, DataFrameSchema

MEASUREMENT_SCHEMA = DataFrameSchema(
    {
        "subject_id": Column(str, Check.str_matches(r"^sub_[a-z0-9]+$")),
        "visit": Column(int, Check.ge(1)),
        "parameter": Column(str, Check.isin(["heart_rate", "spo2"])),
        "value": Column(float, Check.not_equal_to(float("inf")), coerce=True),
        "unit": Column(str, Check.isin(["bpm", "%"])),
        "measured_at": Column("datetime64[ns, UTC]", coerce=True),
    },
    strict=True,
```

```

        coerce=True,
    )

def load_measurements(path: str) -> pd.DataFrame:
    frame = pd.read_csv(path)
    return MEASUREMENT_SCHEMA.validate(frame)

```

La validación de un rango técnico no equivale a interpretar el estado de una persona. Su propósito es detectar errores de captura, formatos imposibles o condiciones definidas por el protocolo de investigación.

Elaborar un informe de calidad

```

from dataclasses import asdict, dataclass
import pandas as pd

@dataclass(frozen=True)
class QualityReport:
    rows: int
    duplicated_rows: int
    missing_by_column: dict[str, int]
    parameters: list[str]

def assess_quality(frame: pd.DataFrame) -> QualityReport:
    return QualityReport(
        rows=len(frame),
        duplicated_rows=int(frame.duplicated().sum()),
        missing_by_column=frame.isna().sum().astype(int).to_dict(),
        parameters=sorted(frame["parameter"].unique().tolist()),
    )

```

El informe debe guardarse junto con el identificador de entrada, la versión del esquema y la fecha del proceso. Corregir una tabla sin conservar el informe original elimina evidencia importante.

Consultar sin cargar todo en memoria

Parquet conserva tipos y facilita lectura por columnas. DuckDB permite consultar archivos analíticos directamente:

```

SELECT
    parameter,
    unit,
    COUNT(*) AS n,
    AVG(value) AS mean_value,
    STDDEV_SAMP(value) AS sd_value
FROM read_parquet('curated/measurements.parquet')
GROUP BY parameter, unit
ORDER BY parameter;

```

Esta alternativa resulta útil cuando el análisis supera el tamaño cómodo de un `DataFrame`, pero todavía no justifica una infraestructura distribuida.

Evitar fuga de información

Si varias filas corresponden al mismo participante, la división entre entrenamiento y prueba debe hacerse por participante, no por fila.

```
from sklearn.model_selection import GroupShuffleSplit

splitter = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=42)
train_idx, test_idx = next(
    splitter.split(frame, groups=frame["subject_id"])
)
```

Mezclar visitas del mismo participante entre conjuntos puede producir resultados artificialmente altos.

Criterio para avanzar

El proceso debe generar una tabla curada en Parquet, un informe de calidad y un registro de procedencia. Una prueba debe demostrar que una unidad inválida o una columna ausente detiene el proceso.

Nivel 3. Trabajar con series de tiempo

Meta: conservar el significado temporal durante la validación, el remuestreo y la generación de ventanas.

Duración sugerida: semanas 12 a 16.

El tiempo es parte del dato

Una serie de tiempo no es una tabla ordenada de manera visual. Cada marca temporal debe representar un instante o intervalo definido. Antes de analizar se debe establecer:

- zona horaria;
- frecuencia esperada;
- tolerancia para retrasos;
- significado de un dato faltante;
- política para duplicados;
- método de interpolación, si se permite; y
- relación entre tiempo del dispositivo y tiempo real.

Validar y ordenar

```
import pandas as pd

def prepare_timeseries(frame: pd.DataFrame) -> pd.DataFrame:
    required = {"subject_id", "timestamp", "value", "unit"}
    missing = required - set(frame.columns)
    if missing:
        raise ValueError(f"Faltan columnas: {sorted(missing)}")

    prepared = frame.copy()
    prepared["timestamp"] = pd.to_datetime(
        prepared["timestamp"], utc=True, errors="raise"
    )
    prepared = prepared.sort_values(["subject_id", "timestamp"])
```

```

duplicated = prepared.duplicated(["subject_id", "timestamp"])
if duplicated.any():
    raise ValueError("Existen marcas temporales duplicadas")

return prepared

```

Remuestrear sin ocultar faltantes

```

def resample_subject(
    frame: pd.DataFrame,
    interval: str = "5min",
) -> pd.DataFrame:
    indexed = frame.set_index("timestamp")
    result = indexed["value"].resample(interval).mean().to_frame()
    result["observed"] = indexed["value"].resample(interval).count().gt(0)
    return result.reset_index()

```

La columna `observed` evita confundir una ventana sin medición con un valor real. Si se interpola, debe agregarse una marca que identifique el valor estimado.

Crear ventanas sin mirar el futuro

Para predecir un evento futuro, las características de una ventana solo pueden utilizar información disponible hasta el momento de la predicción.

```

def rolling_features(series: pd.Series, window: int = 12) -> pd.DataFrame:
    past = series.shift(1)
    return pd.DataFrame(
        {
            "mean_past": past.rolling(window).mean(),
            "std_past": past.rolling(window).std(),
            "min_past": past.rolling(window).min(),
            "max_past": past.rolling(window).max(),
        }
    )

```

El desplazamiento con `shift(1)` impide que la observación que se quiere estimar entre en sus propias características.

Pruebas necesarias

- una fecha sin formato válido debe fallar;
- un duplicado temporal debe detectarse;
- la zona horaria debe normalizarse;
- un intervalo sin observaciones debe conservarse como faltante;
- ninguna característica predictiva debe usar el futuro; y
- el corte entre entrenamiento y prueba debe respetar el orden temporal.

Criterio para avanzar

La plataforma debe generar una serie normalizada, un mapa de faltantes y características por ventana. El informe debe indicar frecuencia esperada, frecuencia observada y política de remuestreo.

Nivel 4. Procesar señales biomédicas

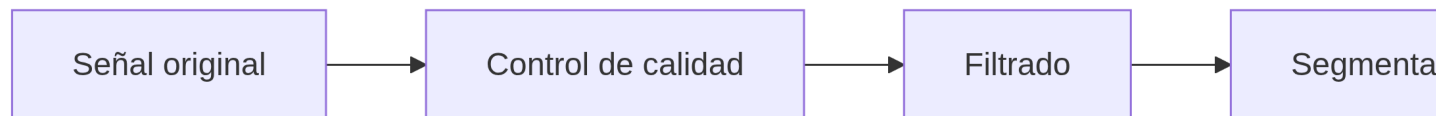
Meta: implementar un flujo de señal que conserve frecuencia de muestreo, canales, unidades y parámetros de procesamiento.

Duración sugerida: semanas 17 a 21.

Contrato específico de señal

Metadato	Ejemplo	Razón
frecuencia de muestreo	500 Hz	relaciona cada muestra con el tiempo
canales	I, II	identifica el origen de cada trayectoria
unidad	mV	permite interpretar amplitud y umbrales
resolución	16 bits	caracteriza la cuantización
dispositivo	simulador-01	conserva procedencia técnica
hora de inicio	ISO 8601	relaciona la señal con otros eventos

Flujo de procesamiento



Cada etapa produce un artefacto nuevo. La señal original no se sobrescribe.

Representación explícita

```
from dataclasses import dataclass
import numpy as np
from numpy.typing import NDArray

@dataclass(frozen=True)
class SignalData:
    samples: NDArray[np.float64]
    sampling_rate_hz: float
    channels: tuple[str, ...]
    unit: str

    @property
    def duration_seconds(self) -> float:
        return self.samples.shape[0] / self.sampling_rate_hz
```

Una matriz sin frecuencia, canales y unidad es insuficiente para representar una señal biomédica.

Diseñar y aplicar un filtro

```
import numpy as np
from scipy.signal import butter, sosfiltfilt

def bandpass_filter(
    signal: SignalData,
    low_hz: float,
    high_hz: float,
    order: int = 4,
) -> SignalData:
    nyquist = signal.sampling_rate_hz / 2
    if not 0 < low_hz < high_hz < nyquist:
        raise ValueError("Las frecuencias de corte no son válidas")

    sos = butter(
        order,
        [low_hz, high_hz],
        btype="bandpass",
        fs=signal.sampling_rate_hz,
        output="sos",
    )
    filtered = sosfiltfilt(sos, signal.samples, axis=0)

    return SignalData(
        samples=np.asarray(filtered, dtype=np.float64),
        sampling_rate_hz=signal.sampling_rate_hz,
        channels=signal.channels,
        unit=signal.unit,
    )
```

El uso de secciones de segundo orden mejora la estabilidad numérica. `sosfiltfilt` evita desfase neto, pero usa muestras futuras y no representa un procesamiento en tiempo real. Esta limitación debe quedar documentada.

Comprobar el filtro

Una prueba sintética puede combinar una componente dentro de la banda y otra fuera de ella. El resultado debe atenuar la segunda sin modificar la longitud ni los metadatos.

```
import numpy as np

def synthetic_signal(fs: float = 500.0, seconds: float = 4.0) -> SignalData:
    time = np.arange(0, seconds, 1 / fs)
    samples = np.sin(2 * np.pi * 10 * time) + 0.5 * np.sin(2 * np.pi * 100 * time)
    return SignalData(samples[:, None], fs, ("synthetic",), "a.u.")
```

Las pruebas con datos sintéticos permiten conocer la respuesta esperada. Después se requieren pruebas con datos representativos del caso de uso.

Artefactos y saturación

El control de calidad puede registrar:

- porcentaje de muestras saturadas;

- intervalos constantes;
- amplitudes no finitas;
- canales ausentes;
- discontinuidades;
- relación señal-ruido estimada; y
- segmentos excluidos y motivo de exclusión.

Una exclusión automática debe ser visible y revisable. Ocultar segmentos defectuosos puede mejorar una métrica sin mejorar la validez del análisis.

Criterio para avanzar

El motor de señales debe leer un ejemplo sintético, validar metadatos, aplicar un proceso parametrizado y guardar tanto el resultado como la procedencia. Las pruebas deben confirmar longitud, canales, unidad y respuesta frecuencial aproximada.

Nivel 5. Procesar imágenes biomédicas

Meta: leer y transformar imágenes sin perder escala espacial, orientación, intensidad ni procedencia.

Duración sugerida: semanas 22 a 26.

Una imagen no es solo una matriz

En una fotografía común puede ser suficiente conocer alto, ancho y canales. En una imagen médica se requieren además:

- modalidad de adquisición;
- tamaño físico del píxel o vóxel;
- orientación y posición;
- número de cortes y orden espacial;
- profundidad y representación de intensidad;
- transformación aplicada por el equipo; y
- identificadores que deban anonimizarse.

Leer DICOM sin perder metadatos

```
from dataclasses import dataclass
from pathlib import Path

import numpy as np
import pydicom
from numpy.typing import NDArray

@dataclass(frozen=True)
class MedicalImage2D:
    pixels: NDArray[np.float32]
    pixel_spacing_mm: tuple[float, float] | None
    modality: str
    source_sop_instance_uid: str

def load_dicom_image(path: Path) -> MedicalImage2D:
    dataset = pydicom.dcmread(path)
```

```

pixels = dataset.pixel_array.astype(np.float32)

slope = float(getattr(dataset, "RescaleSlope", 1.0))
intercept = float(getattr(dataset, "RescaleIntercept", 0.0))
pixels = pixels * slope + intercept

spacing_raw = getattr(dataset, "PixelSpacing", None)
spacing = None
if spacing_raw is not None:
    spacing = (float(spacing_raw[0]), float(spacing_raw[1]))

return MedicalImage2D(
    pixels=pixels,
    pixel_spacing_mm=spacing,
    modality=str(dataset.Modality),
    source_sop_instance_uid=str(dataset.SOPInstanceUID),
)

```

Este ejemplo es educativo. Una implementación real debe verificar transferencia, compresión, fotometría, orientación, múltiples marcos y metadatos específicos de la modalidad.

PixelSpacing no es resolución de pantalla

PixelSpacing expresa la distancia física entre centros de píxeles en el paciente, normalmente en milímetros. No equivale a píxeles por pulgada ni debe modificarse solo para aumentar el tamaño visual de una imagen.

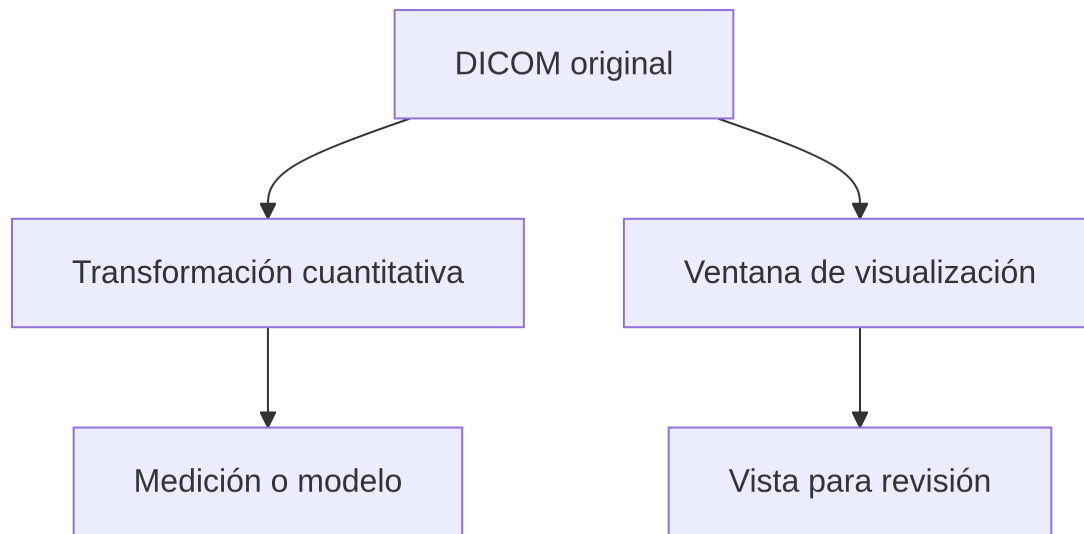
Cuando una imagen se remuestrea, la matriz y el espaciamiento deben cambiar de manera coordinada para conservar el tamaño físico:

$$N_{nuevo} \times s_{nuevo} \approx N_{original} \times s_{original}$$

donde N es el número de píxeles en un eje y s es el espaciamiento físico.

ImagerPixelSpacing, cuando existe, describe el plano del detector. **PixelSpacing** describe el espaciamiento en el paciente después de las correcciones geométricas aplicables. No son intercambiables.

Separar visualización y cuantificación



Una ventana de contraste puede mejorar la visualización sin cambiar los datos cuantitativos. Si se guarda una imagen PNG para presentación, debe identificarse como derivado y no como sustituto del DICOM original.

Anonimización y riesgo residual

Eliminar algunos campos de cabecera no garantiza anonimización. Pueden existir:

- identificadores privados del fabricante;
- texto incrustado en los píxeles;
- fechas que permitan reidentificación;
- identificadores dentro de objetos relacionados; y
- rasgos anatómicos identificables en ciertos estudios.

La anonimización requiere un perfil definido, verificación y control institucional. En el tutorial se utilizan imágenes sintéticas o conjuntos abiertos con condiciones de uso revisadas.

Pruebas necesarias

- la lectura conserva forma, tipo y modalidad;
- la escala de intensidad se aplica de manera conocida;
- el espaciamiento se interpreta en el orden correcto;
- un remuestreo conserva aproximadamente el tamaño físico;
- la orientación de un volumen no cambia sin registro; y
- cada derivado referencia el objeto fuente.

Criterio para avanzar

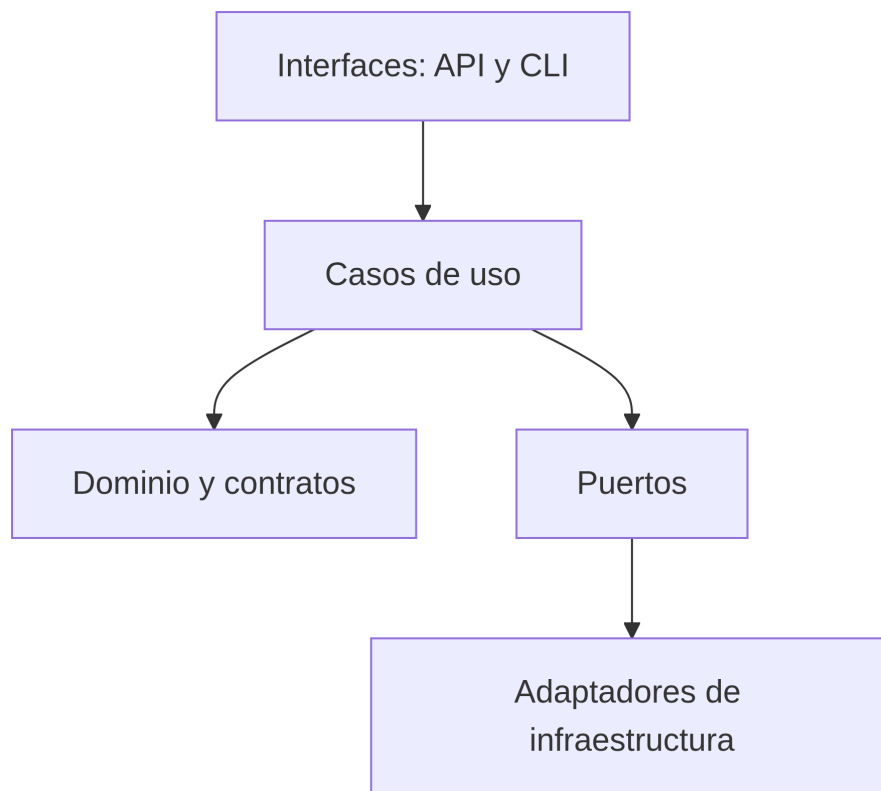
El motor de imágenes debe producir una representación cuantitativa y una vista de revisión, ambas enlazadas al original. El manifiesto debe conservar escala física, modalidad y transformación.

Nivel 6. Integrar la plataforma

Meta: conectar los motores mediante casos de uso, persistencia, API y ejecución de trabajos sin acoplar el dominio a una tecnología específica.

Duración sugerida: semanas 27 a 31.

Separar responsabilidades



Capa	Responsabilidad	No debería conocer
Dominio	conceptos y reglas estables	FastAPI, SQLAlchemy o rutas del servidor
Aplicación	coordinar casos de uso	detalles del motor de base de datos
Infraestructura	implementar almacenamiento y mensajería	decisiones de presentación
Interfaces	recibir solicitudes y presentar respuestas	lógica científica interna

Definir un procesador común

```
from pathlib import Path
from typing import Protocol

from bioforge.domain.models import DatasetManifest, ProcessRecord

class ModalityProcessor(Protocol):
    def supports(self, manifest: DatasetManifest) -> bool: ...

    def process(
        self,
        manifest: DatasetManifest,
```



```

        source: Path,
        parameters: dict[str, object],
    ) -> ProcessRecord: ...

```

Cada motor implementa el mismo contrato. La plataforma selecciona el procesador por capacidad, no mediante una cadena creciente de condiciones distribuidas por todo el código.

Caso de uso de análisis

```

class RunAnalysis:
    def __init__(self, catalog, storage, processors):
        self.catalog = catalog
        self.storage = storage
        self.processors = processors

    def execute(self, dataset_id: str, parameters: dict[str, object]):
        manifest = self.catalog.get_dataset(dataset_id)
        source = self.storage.materialize(manifest.source_uri)

        processor = next(
            (item for item in self.processors if item.supports(manifest)),
            None,
        )
        if processor is None:
            raise ValueError(f"Modalidad no soportada: {manifest.modality}")

        return processor.process(manifest, source, parameters)

```

Este caso de uso coordina, pero no contiene fórmulas de filtrado, consultas SQL ni lectura de DICOM.

API mínima

Método y ruta	Función
POST /datasets	registrar un manifiesto
GET /datasets/{dataset_id}	consultar metadatos
POST /datasets/{dataset_id}/analyses	solicitar un análisis
GET /processes/{process_id}	consultar estado y advertencias
GET /artifacts/{artifact_id}	obtener metadatos de un resultado
GET /health	comprobar disponibilidad técnica

```

from fastapi import APIRouter, Depends, status

from bioforge.domain.models import DatasetManifest

router = APIRouter(prefix="/datasets", tags=["datasets"])

@router.post("", status_code=status.HTTP_201_CREATED)
def register_dataset(
    manifest: DatasetManifest,
    service=Depends(get_dataset_service),
):
    return service.register(manifest)

```

FastAPI genera un contrato OpenAPI, pero la documentación automática no reemplaza la explicación del significado de cada campo, sus unidades y sus restricciones.

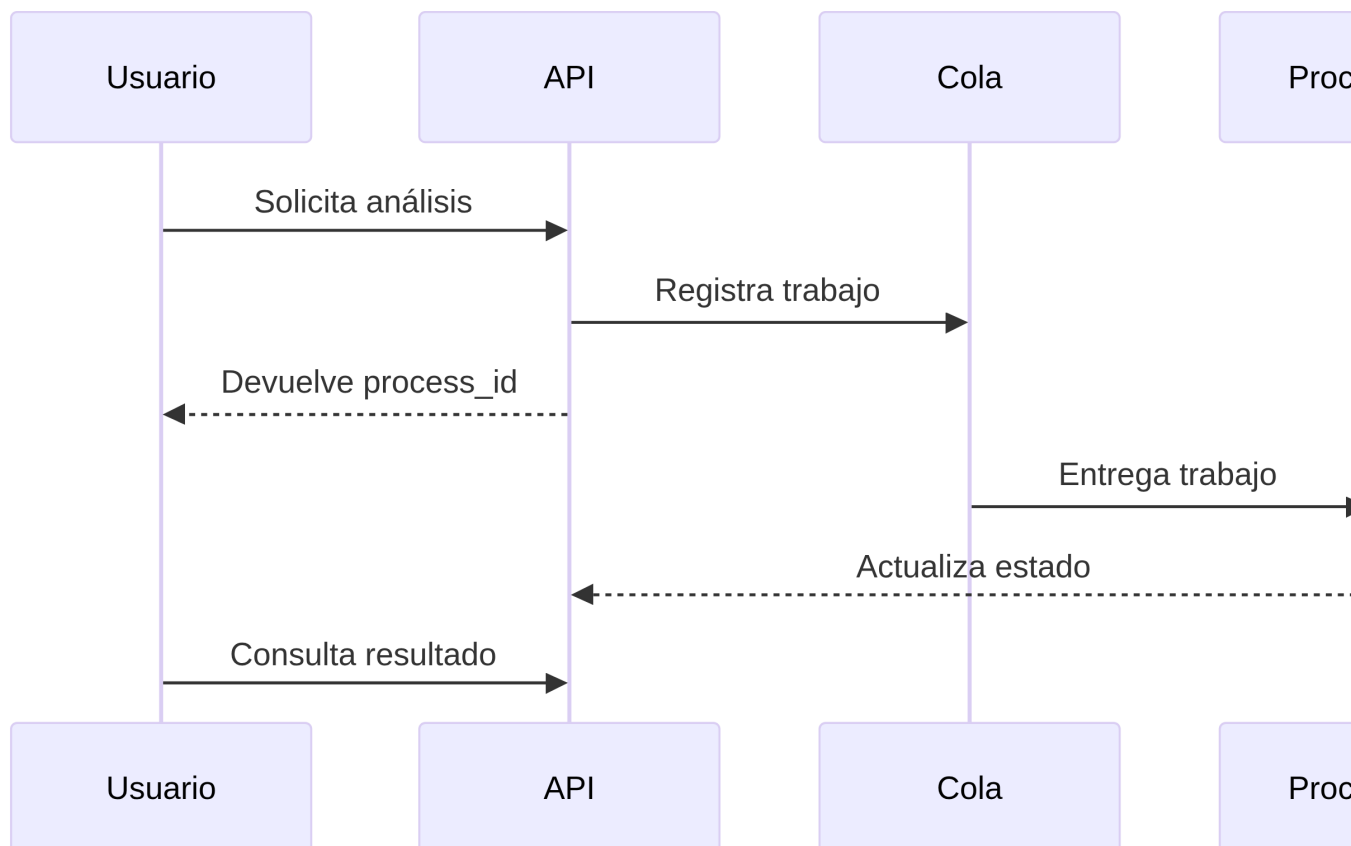
Persistencia apropiada

Información	Almacenamiento sugerido	Motivo
catálogos, usuarios y procesos	PostgreSQL	relaciones, transacciones y consultas
archivos originales	almacenamiento de objetos	tamaño, inmutabilidad y acceso controlado
derivados voluminosos	almacenamiento de objetos	evita inflar la base relacional
caché temporal	memoria o Redis	acelera operaciones repetidas no críticas
eventos técnicos	sistema de registros	búsqueda y correlación operacional

Guardar un archivo DICOM, un volumen o una señal extensa dentro de una tabla relacional puede ser válido en casos concretos, pero no debe ser la decisión por defecto.

Trabajos asíncronos

El registro de metadatos puede responder en milisegundos. Un análisis de imágenes puede tardar minutos. La API no debería mantener abierta la solicitud durante todo el proceso.



La cola se introduce cuando existe una necesidad real de desacoplar el tiempo de respuesta. Para la primera versión, un ejecutor local puede implementar el mismo puerto.

Criterio para avanzar

Una solicitud debe registrar un conjunto, seleccionar el motor correcto, crear un proceso, producir un artefacto y permitir consultar su estado. El flujo debe probarse de extremo a extremo con datos sintéticos de las cuatro modalidades.

Nivel 7. Preparar la plataforma para operar

Meta: hacer que el sistema pueda desplegarse, observarse y mantenerse sin perder seguridad ni reproducibilidad.

Duración sugerida: semanas 32 a 36.

Estrategia de pruebas

Tipo	Qué verifica	Ejemplo
Unitaria	una regla aislada	un filtro rechaza cortes por encima de Nyquist
Propiedades	invariantes para muchas entradas	remuestrear conserva tamaño físico dentro de tolerancia
Integración	interacción entre componentes	API registra el manifiesto en la base
Contrato	compatibilidad de una interfaz	respuesta cumple el esquema OpenAPI
Extremo a extremo	flujo completo	archivo válido produce un artefacto trazable
Rendimiento	tiempo y memoria	volumen de prueba se procesa dentro del límite
Seguridad	controles y dependencias	usuario sin permiso no descarga un original
Validación científica	comportamiento del método	resultado concuerda con una referencia conocida

Una cobertura alta no demuestra validez científica. Tampoco una comparación con una referencia sustituye las pruebas de seguridad, integración o permisos.

Integración continua

```
name: quality

on:
  pull_request:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
```

```

- uses: actions/checkout@v4
- uses: astral-sh/setup-uv@v6
- run: uv sync --locked --dev
- run: uv run ruff check .
- run: uv run ruff format --check .
- run: uv run pyright
- run: uv run pytest --cov=src --cov-report=term-missing

```

En una organización se deben fijar las acciones a revisiones verificadas y aplicar controles de cadena de suministro. El ejemplo se mantiene breve para explicar el flujo.

Contenedor reproducible

```

FROM python:3.13-slim AS builder

COPY --from=ghcr.io/astral-sh/uv:latest /uv /usr/local/bin/uv
WORKDIR /app

COPY pyproject.toml uv.lock ./
RUN uv sync --locked --no-dev --no-install-project

COPY src ./src
RUN uv sync --locked --no-dev

FROM python:3.13-slim
RUN useradd --create-home --uid 10001 appuser
WORKDIR /app

COPY --from=builder /app /app
USER appuser

ENV PATH="/app/.venv/bin:$PATH"
EXPOSE 8000

CMD ["uvicorn", "bioforge.interfaces.api:app", "--host", "0.0.0.0", "--port", "8000"]

```

En producción se debe fijar la imagen base por *digest*, escanear dependencias, generar un inventario de componentes y definir un proceso de actualización.

Seguridad desde el diseño

Controles mínimos:

- autenticación separada de la lógica de análisis;
- autorización por recurso, rol y propósito;
- cifrado en tránsito y en reposo;
- secretos fuera del repositorio y de la imagen del contenedor;
- límites de tamaño, tipo y frecuencia de carga;
- nombres internos generados por el sistema;
- validación de archivos antes de procesarlos;
- registros sin datos biomédicos sensibles;
- dependencias bloqueadas y revisadas;
- respaldo y prueba de restauración; y
- retención y eliminación definidas por política.

Los archivos deben considerarse entradas no confiables. Un nombre válido o una extensión conocida no demuestra que el contenido sea seguro.

Observabilidad

La observabilidad permite comprender el estado interno mediante salidas del sistema. [OpenTelemetry](#) organiza esta información principalmente como trazas, métricas y registros.

Señal operacional	Pregunta
Registros	¿qué evento ocurrió y con qué nivel?
Métricas	¿cuánto tarda, cuántas veces falla y qué recursos consume?
Trazas	¿por cuáles componentes pasó una solicitud?

Métricas útiles para BioForge Analytics:

- trabajos por modalidad;
- tiempo de espera y procesamiento;
- porcentaje de fallos por etapa;
- memoria máxima por trabajo;
- archivos rechazados por regla de calidad;
- disponibilidad de la API; y
- antigüedad del trabajo pendiente más viejo.

No use `subject_id`, diagnóstico, ruta original o contenido del estudio como etiqueta de una métrica. Además del riesgo de privacidad, las etiquetas de alta cardinalidad degradan el sistema de monitoreo.

Interoperabilidad

Interoperar no es exportar cualquier JSON. Se necesita una definición compartida de estructura, terminología, identidad, versión y reglas de intercambio.

- [HL7 FHIR](#) proporciona un estándar para intercambiar información de salud.
- [DICOMweb](#) define servicios web para consultar, almacenar y recuperar recursos DICOM.
- En Colombia, la estrategia de [Interoperabilidad de la Historia Clínica Electrónica](#) adopta lineamientos para el intercambio seguro y estandarizado de información clínica.

La plataforma interna puede usar un modelo propio para análisis, pero debe incorporar una capa de mapeo cuando intercambie información con sistemas externos. El modelo interno no debe declararse FHIR o DICOM solo porque utiliza nombres parecidos.

Criterio para avanzar

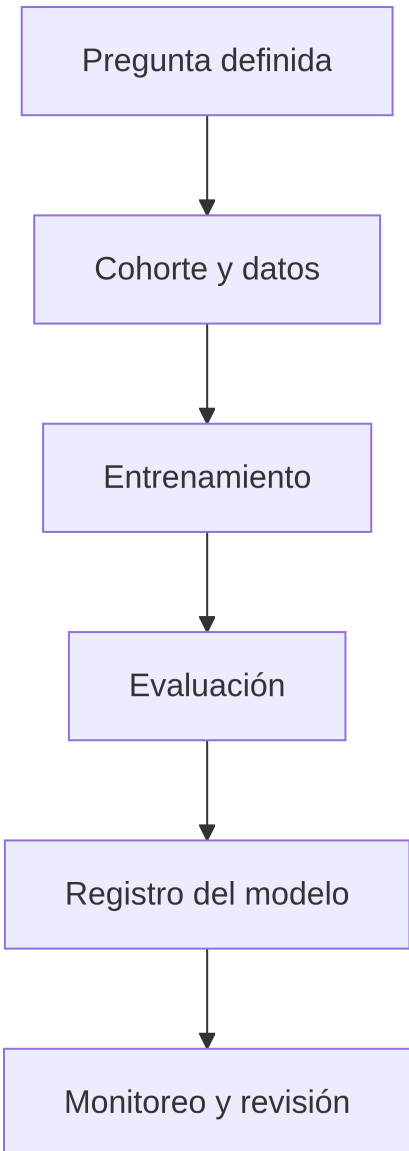
La plataforma debe desplegarse de forma repetible, rechazar accesos no autorizados, registrar telemetría sin datos sensibles y recuperar su catálogo desde un respaldo probado.

Nivel 8. Analítica avanzada e inteligencia artificial

Meta: incorporar modelos sin romper la trazabilidad, la separación por participante ni la responsabilidad humana.

Duración sugerida: semanas 37 a 40.

El modelo es un componente, no la plataforma



Un modelo debe registrar:

- propósito y población prevista;
- definición de entrada y salida;
- versión del conjunto de datos;
- división por participante y, cuando corresponda, por tiempo o institución;
- código, parámetros y semilla;
- métricas con intervalos de incertidumbre;
- análisis por subgrupos relevantes y justificados;
- errores conocidos y condiciones de exclusión;
- versión del entorno; y
- criterio para suspender o actualizar el modelo.

Registro mínimo del modelo

```
model_id: mdl_fall_risk_001
task: classification
intended_use: research_only
inputs:
  - tabular_features_v3
  - gait_signal_features_v2
target_definition: outcome_at_6_months_v1
data_split:
  unit: subject
  strategy: grouped_temporal_holdout
metrics:
  roc_auc: 0.81
  sensitivity_at_threshold: 0.76
  specificity_at_threshold: 0.73
limitations:
  - single-center synthetic example
  - not validated for clinical decisions
```

Una métrica aislada no permite juzgar utilidad. El umbral, la prevalencia, los costos de error, la calibración y el contexto de uso cambian la interpretación.

Analítica multimodal

La integración de modalidades puede realizarse en diferentes momentos:

Estrategia	Descripción	Ventaja	Riesgo
Fusión temprana	combina variables antes del modelo	implementación sencilla con características compatibles	escalas y faltantes pueden dominar
Fusión intermedia	combina representaciones aprendidas	captura relaciones complejas	exige más datos y validación
Fusión tardía	combina predicciones de modelos separados	módulos independientes y fáciles de comparar	puede perder interacción entre modalidades

Se debe construir primero una línea base por modalidad. Un modelo multimodal solo se justifica si agrega valor medible y estable.

Causalidad, explicabilidad y confianza

La plataforma puede apoyar análisis causal, pero un modelo predictivo no se vuelve causal por incluir muchas variables. Se requiere expresar supuestos, definir el estimando, representar relaciones plausibles y evaluar amenazas como confusión, selección y medición.

De igual manera, una gráfica de importancia de variables no constituye por sí sola una explicación suficiente para una persona afectada. La explicación debe responder al propósito, el contexto y las consecuencias de la decisión.

Uso de IA para desarrollar la plataforma

Los asistentes de IA pueden ayudar a:

- proponer pruebas a partir de un requisito;

- explicar un error y sugerir hipótesis;
- generar datos sintéticos pequeños;
- comparar alternativas de diseño;
- redactar documentación inicial; y
- revisar consistencia entre contrato, código y prueba.

Todo resultado debe revisarse. No se deben compartir datos personales, secretos, credenciales o archivos clínicos con un servicio sin autorización institucional y contrato apropiado.

Una solicitud técnica útil incluye contexto y restricciones:

Implemente una función Python que valide la frecuencia de muestreo de una señal. Debe rechazar valores no finitos, menores o iguales a cero y frecuencias incompatibles con un corte de 100 Hz. Use tipos, mensajes de error claros y pruebas con pytest. No agregue dependencias.

Monitorear después del despliegue

Elemento	Ejemplo de cambio
Datos	nuevo dispositivo, unidad o protocolo
Calidad	aumento de canales ausentes
Distribución	cambio en edades, intensidades o frecuencias
Rendimiento	menor sensibilidad o peor calibración
Operación	más latencia, memoria o fallos
Uso	usuarios aplican el resultado fuera del alcance

El monitoreo técnico no sustituye el seguimiento clínico ni la revisión humana.

Criterio de finalización

El modelo debe poder reconstruirse desde datos versionados, código, configuración y entorno. Su evaluación debe respetar la unidad de participante, declarar incertidumbre y documentar límites. La plataforma debe permitir retirar una versión sin perder el historial.

Tendencias actuales y criterios de adopción

Una tendencia es útil cuando resuelve una limitación demostrada. La siguiente tabla evita adoptar herramientas solo por novedad.

Tendencia	Aplicación razonable	Señal de adopción prematura
Formatos columnares	Parquet para tablas grandes y analítica por columnas	convertir archivos pequeños sin beneficio medido
Motores embebidos	DuckDB para consultar Parquet local o en objetos	desplegar un clúster para una carga moderada
Arquitectura <i>lakehouse</i>	datos diversos, historial y múltiples consumidores	ausencia de catálogo, gobierno y volumen real
Arreglos fragmentados	Zarr para volúmenes y matrices multidimensionales	usarlo sin definir acceso por bloques
Cómputo asíncrono	análisis que tarda más que una solicitud web	agregar una cola para operaciones inmediatas
GPU	imágenes, aprendizaje profundo o operaciones paralelas medidas	mover todo el preprocesamiento a GPU

Tendencia	Aplicación razonable	Señal de adopción prematura
MLOps	varios modelos, versiones y monitoreo repetido	automatizar un modelo que todavía cambia cada día
IA generativa	documentación, búsqueda controlada o apoyo analítico	delegar decisiones o introducir datos sensibles
Estándares abiertos	intercambio con sistemas de salud	mapear sin perfiles, terminologías o validación
Observabilidad abierta	correlacionar fallos entre componentes	recopilar telemetría sin una pregunta operacional
Monolito modular	equipo pequeño y dominio en evolución	mezclar todas las reglas en un solo módulo

Matriz de decisión

Antes de incorporar una tecnología, responda:

1. ¿Qué problema medido resuelve?
2. ¿Qué alternativa más sencilla existe?
3. ¿Cómo se probará su beneficio?
4. ¿Qué nuevos riesgos introduce?
5. ¿Quién podrá mantenerla?
6. ¿Cómo se retira si no funciona?

Si estas preguntas no tienen respuesta, la decisión aún no está madura.

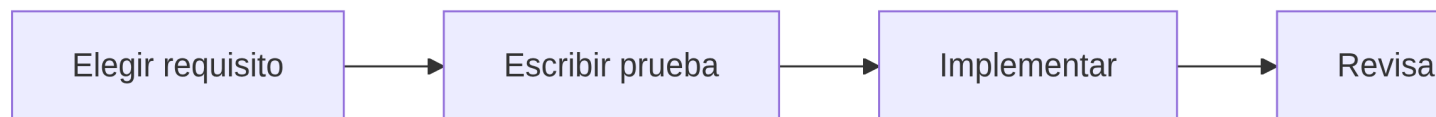
Efectividad y eficiencia del desarrollador

La productividad no se mide por líneas de código. En esta plataforma se mide por la capacidad de entregar cambios pequeños, correctos, revisables y útiles.

Hábitos de alto impacto

- comenzar por un requisito y un ejemplo verificable;
- dividir el trabajo en incrementos que puedan revisarse;
- automatizar tareas repetidas después de comprenderlas;
- mantener funciones pequeñas y nombres precisos;
- ejecutar pruebas antes de integrar cambios;
- registrar decisiones que tengan consecuencias futuras;
- medir antes de optimizar;
- conservar datos de muestra sintéticos y representativos;
- revisar errores por causa, no solo corregir el síntoma; y
- eliminar código y dependencias que dejaron de aportar valor.

Flujo diario breve



Indicadores del proceso

Indicador	Interpretación prudente
tiempo desde cambio hasta validación	detecta demoras del flujo
frecuencia de integración	favorece incrementos pequeños
porcentaje de cambios que fallan	muestra estabilidad, no culpables
tiempo de recuperación	mide capacidad de responder a fallos
defectos encontrados antes de producción	evalúa controles preventivos
tiempo para reproducir un análisis	mide calidad de entorno y documentación

Las métricas deben orientar mejoras del sistema de trabajo. Utilizarlas para clasificar personas favorece comportamientos que distorsionan el indicador.

Plan de estudio de 40 semanas

Semanas	Tema	Producto verificable
1-2	necesidad, alcance, Git y uv	repositorio reproducible y ADR inicial
3-4	Python, tipos y funciones	modelos de dominio y validaciones básicas
5-6	pruebas y procedencia	integridad de archivos y registro de procesos
7-8	contratos tabulares	esquema y datos sintéticos válidos e inválidos
9-11	calidad y consulta tabular	Parquet, informe y consulta DuckDB
12-13	tiempo, zonas y duplicados	serie normalizada
14-16	remuestreo y ventanas	mapa de faltantes y características sin fuga
17-18	muestreo y representación de señales	contrato de señal y prueba sintética
19-21	filtrado, segmentos y características	proceso de señal trazable
22-23	DICOM y metadatos espaciales	lector validado
24-26	remuestreo, visualización y derivados	artefactos de imagen trazables
27-28	capas, puertos y adaptadores	motores conectados al núcleo
29-31	API, persistencia y trabajos	flujo completo por modalidad
32-33	pruebas de sistema y seguridad	matriz de riesgos y controles
34-36	contenedor, CI y observabilidad	despliegue diagnosticable
37-38	modelo base y evaluación	experimento reconstruible
39	interoperabilidad	mapeo documentado y validado
40	cierre y demostración	versión etiquetada y evaluación final

Portafolio que demuestra capacidad

El portafolio final debe permitir evaluar el proceso, no solo observar capturas de pantalla.

Incluya:

1. descripción del problema y de los usuarios;
2. alcance y exclusiones explícitas;
3. diagrama de arquitectura;
4. contratos de datos por modalidad;
5. datos sintéticos de prueba;
6. instrucciones reproducibles de instalación;
7. pruebas y reporte de calidad;
8. ejemplo completo desde original hasta artefacto;
9. decisiones de arquitectura y sus consecuencias;
10. amenazas de seguridad y controles;
11. resultados de rendimiento;
12. límites científicos y clínicos;
13. plan de monitoreo; y
14. demostración breve de la API o interfaz.

Rúbrica de autoevaluación

Criterio	Inicial	Competente	Avanzado
Problema	describe una idea general	define usuario, tarea y criterio	relaciona necesidad, riesgo y evidencia
Datos	procesa un archivo	valida esquema y metadatos	conserva linaje entre modalidades
Código	funciona localmente	es modular, tipado y probado	permite extender motores sin romper contratos
Ciencia	calcula resultados	documenta parámetros y supuestos	valida contra referencias y declara incertidumbre
Seguridad	protege credenciales	aplica permisos y validación	verifica controles y recuperación
Operación	se ejecuta manualmente	se despliega de forma repetible	se observa y recupera ante fallos
Comunicación	muestra resultados	explica decisiones y límites	adapta evidencia a perfiles técnicos y de salud

Lista final de verificación

Problema y alcance

- ☐ La plataforma responde a una necesidad concreta.
- ☐ Los usuarios y las tareas están definidos.
- ☐ El uso previsto y los usos excluidos son explícitos.
- ☐ Los riesgos de error, demora e indisponibilidad están descritos.

Datos y modalidades

- ☐ Cada conjunto tiene identificador, origen, checksum y versión de esquema.
- ☐ Las tablas conservan tipos, unidades y categorías.
- ☐ Las series conservan zona horaria, frecuencia y marcas de faltantes.
- ☐ Las señales conservan muestreo, canales, unidades y parámetros.
- ☐ Las imágenes conservan escala, orientación, modalidad e intensidad.
- ☐ Los archivos originales no se sobrescriben.

Código y arquitectura

- ☐ El entorno se reconstruye desde archivos versionados.
- ☐ Dominio, casos de uso e infraestructura están separados.
- ☐ Cada motor implementa un contrato común.
- ☐ Las dependencias entre módulos son explícitas.
- ☐ Las decisiones importantes tienen un ADR.

Calidad, seguridad y operación

- ☐ Existen pruebas unitarias, de integración y extremo a extremo.
- ☐ La validación científica se diferencia de la prueba de software.
- ☐ Los permisos se verifican por recurso y acción.
- ☐ Los registros no contienen información sensible.
- ☐ El respaldo fue restaurado en una prueba.
- ☐ Las métricas permiten localizar fallos sin exponer participantes.

Analítica e IA

- ☐ La división de datos respeta participantes y tiempo.
- ☐ Existe una línea base antes del modelo complejo.
- ☐ Las métricas se relacionan con el uso previsto.
- ☐ Se documentan incertidumbre, límites y subgrupos relevantes.
- ☐ El modelo puede retirarse o reemplazarse de forma controlada.
- ☐ La decisión final conserva revisión humana cuando corresponde.

Fuentes y documentación recomendada

Desarrollo y datos

- [Python](#)
- [uv](#)
- [Quarto](#)
- [Pydantic](#)
- [Pandera](#)
- [Apache Parquet](#)
- [DuckDB](#)
- [xarray](#)
- [SciPy Signal](#)
- [pydicom](#)
- [SimpleITK](#)

Calidad, seguridad y operación

- [pytest](#)
- [OWASP Top 10:2025](#)
- [OpenTelemetry](#)
- [DORA 2025](#)
- [OpenSSF](#)

Salud digital e interoperabilidad

- [HL7 FHIR](#)
- [DICOM Standard](#)
- [DICOMweb](#)

- [Interoperabilidad de la Historia Clínica Electrónica en Colombia](#)

Cierre

Construir una plataforma de análisis biomédico no significa reunir todas las tecnologías disponibles. Significa crear un camino confiable desde el dato original hasta un resultado que pueda explicarse, verificarse y reproducirse.

Las tablas, las series, las señales y las imágenes requieren tratamientos diferentes, pero comparten una misma obligación: conservar su significado. Cuando la arquitectura mantiene contratos claros, procedencia, pruebas, seguridad y revisión humana, la plataforma deja de ser un conjunto de scripts y se convierte en infraestructura de conocimiento para la investigación y el cuidado de la salud.